Department of Computer Science and Engineering

Assignment Questions

Course Code & Name: CSA09 – Programming in Java (SLOT B)

Assignment Title:	**Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System using Java**
Course Outcome(s) – CO:	**CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations.** **CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.** **CO3: Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques.**
Bloom's Taxonomy Level	**L2 – Understand; L3 – Apply; L4 – Analyse**
SDG Mapping (SDG 1-17)	**SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure; SDG 10 – Reduced Inequalities**

Problem Statement : Design, implement, and evaluate a menu-driven Java application to manage a Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System, where patients book appointments and are prescribed medicines, medicine stock is a limited resource that must be tracked accurately, and appointment-fulfilment activities must be handled reliably. The application shall automate the workflow — registering patients and doctors/medicines, displaying available appointment slots and medicine stock, booking and cancelling appointments, searching and updating patient/medicine records, maintaining a waitlist for fully booked doctors, generating patient-visit and pharmacy-inventory reports, and sending appointment-reminder and low-stock notifications. Core entities such as Patient, Doctor/Appointment, Medicine, Inventory, and Notification shall be modelled as encapsulated classes with suitable data members and member methods. Different patient categories or medicine types shall be organized through inheritance and polymorphism so that consultation-fee waivers, dosage rules, or notification behaviour can adapt at run time. Java Collection Framework classes such as ArrayList, Set, HashMap, and Hashtable shall be used with generics and Iterator/List Iterator for storing, retrieving, updating, and traversing records.The system shall use built-in and user-defined exception handling for invalid patient IDs, duplicate appointments, out-of-stock medicines, invalid input, and other exceptional conditions. Multithreading shall be incorporated to simulate concurrent appointment-booking and notification- dispatch tasks, using thread priorities, synchronization, and inter-thread communication to safely update shared appointment-slot and inventory information. The program shall provide a clean menu-driven interface and generate a meaningful patient-visit and inventory-utilization report.

SMART HOSPITAL PATIENT APPOINTMENT, PHARMACY INVENTORY AND ALERT NOTIFICATION SYSTEM USING JAVA

ASSIGNMENT REPORT

Submitted by:

N. VINSENT 192425202

S.TANVEER MUHAMMED 192311296

A. SULOCHANA 192372370

A. BHARGAV REDDY 192465061

Under the Supervision of

Thalapathi Rajasekaran R

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Abstract

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System is a menu driven Java application designed to gather everyday hospital tasks into one small easy to use software system. The project focuses on registering patients managing doctors and appointments tracking medicine stock writing prescriptions handling waitlists and creating alerts. The main idea is to replace scattered records with organized objects and collections so that information can be searched, updated and reported consistently.The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System is deliberately built around the Java concepts covered in the course. Encapsulation is used to keep patient, doctor, appointment and medicine data safe. Inheritance and polymorphism are used for patient categories and medicine behavior. ArrayList, Set, HashMap and Hashtable are used to keep collections of records. Iterators are used while going through and updating collections. User defined exceptions help spot operations while multithreading shows how booking and notification tasks can run at the same time. Synchronization is applied to shared appointment and inventory resources so that two operations do not accidentally take the slot or stock item.The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System is meant as a prototype rather than a real hospital information system. Its value lies in showing how object oriented programming, collections, exception handling and concurrency can be combined into an application. Test cases cover exceptional operations such as duplicate booking, invalid patient identification, out of stock medicine and concurrent appointment requests. The final design aims for a balance, between simplicity, readability and enough functionality to show the course outcomes.

1. Problem Statement and Problem Formulation

Hospitals deal with appointments, patient records and medicines every single day. In a small clinic things can get busy fast. Appointment slots fill up quickly. Medicines run low. Patients might forget their visits. When all of this is managed in systems errors creep in—like double bookings, wrong medicine counts or missed follow-ups. That's why this assignment asks for a Java application that brings together the main parts of a patient's visit into one system. The problem is about managing resources across three key areas. First doctor appointment slots must be assigned properly. No two patients can have the time slot. Second the pharmacy stock must stay accurate. Every time a medicine is prescribed or given the system should reduce the count. It must also alert users when a medicine is near its reorder point. Third some tasks happen at the time—like when a patient tries to book an appointment or when a reminder is sent. These need to run at the time but without breaking anything especially when they use shared data. The system must support the basic flow. That includes adding patients and doctors adding medicines showing available appointments and current stock, booking or canceling visits putting patients on a waitlist if the doctor is full, searching and updating records writing prescriptions generating reports and sending alerts, for low stock or upcoming appointments. It must also give messages when something can't be done—like trying to book a time that is already taken or checking out a medicine that's not in stock.

Problem formulation

- Input: details, doctor details, appointment requests, medicine details, prescription quantities and user menu choices.
- Processing: The system validates details, doctor details, appointment requests, medicine details, prescription quantities and user menu choices. The system allocates appointment slots maintains waitlists updates stock creates notifications and produces reports.
- Output: The system provides booking confirmations, cancellation messages, alerts, inventory status, patient-visit reports and pharmacy-utilization reports.
- Safety condition: The system ensures appointment slots and medicine quantities remain consistent when operations are executed by multiple threads.
- Error condition: The system reports IDs, duplicate appointments, unavailable slots and insufficient stock, without terminating the application..

2. Objective and Expected Outcomes

Objectives

- To design a realistic hospital-management problem using object-oriented Java.
- To demonstrate encapsulation, inheritance and polymorphism through hospital entities.
- To use Java Collection Framework classes with generics for record management.
- To implement exception handling for invalid and exceptional operations.
- To demonstrate multithreading, synchronization and inter-thread communication.
- To maintain reliable appointment-slot and medicine-stock information.
- To generate useful patient-visit and pharmacy-inventory reports.
- To create appointment-reminder and low-stock notification behaviour.

Expected outcomes

After implementation, a user should be able to complete a typical workflow without editing program data manually. A patient can be registered, an available appointment can be selected, the booking can be confirmed and a reminder can be generated. A doctor can be assigned to a patient, medicines can be prescribed and inventory can be updated. If a doctor is fully booked, the patient can enter a waitlist instead of being lost. If stock falls below the configured threshold, a notification is produced.

From the academic perspective, the project should provide visible evidence for CO1 through the class hierarchy and object interactions, CO2 through collections and iterators, and CO3 through threads, synchronization, communication and exception handling.

3. Requirements, Constraints and Assumptions

Functional requirements

- Patient registration, search and update.
- Doctor registration and appointment-slot management.
- Appointment booking, cancellation and waitlist management.
- Medicine registration, search, update and stock display.
- Prescription entry and controlled inventory deduction.
- Appointment-reminder and low-stock notifications.
- Patient-visit and inventory-utilization reports.
- Menu-driven console interaction.

- Concurrent booking and notification simulation.

Non-functional requirements

- Readable and modular Java code.
- Consistent shared-state updates.
- Meaningful error messages.
- Use of generics instead of raw collections.
- Reasonable response time for a small in-memory dataset.
- Simple console interface suitable for demonstration.

Constraints

The prototype stores data in memory, so information is lost when the program closes unless file or database persistence is added. It is not designed for multiple hospital branches, network users, online payment, electronic health-record standards or real clinical decision-making. The notification mechanism is simulated through console messages. Appointment and medicine rules are simplified for academic demonstration.

Assumptions

- Each patient and doctor has a unique ID.
- Each medicine has a unique medicine ID.
- A doctor cannot have two patients in the same appointment slot.
- Stock quantity cannot become negative.
- A cancellation releases the appointment slot.
- The waitlist follows first-come, first-served order.
- Reminder and low-stock messages are treated as simulated notifications.
- Dates and times are entered in a simple, validated format.

4. Application of Relevant Course Knowledge / Concepts

Course concept	Application in project	Evidence
Classes and objects	Patient, Doctor, Appointment, Medicine, Inventory, Notification and Report classes.	Objects represent individual hospital records.
Encapsulation	Private fields with getters/setters and validation methods.	Data cannot be modified directly.
Inheritance	Patient subtypes such as RegularPatient and SeniorPatient; medicine subtypes	Common attributes are reused.

	if required.	
Polymorphism	Overridden fee/notification/dosage behaviour.	Runtime object type determines behaviour.
ArrayList	Appointments, waitlists and notification history.	Ordered records.
Set	Unique patient/doctor/medicine identifiers.	Duplicate IDs are rejected.
HashMap	Fast lookup of patients, doctors and medicines by ID.	Key-value record storage.
Hashtable	Thread-safe legacy-style notification/lookup demonstration.	Synchronized map operations.
Iterator/ListIterator	Traversal and safe update/removal of collection entries.	Collection processing.
Generics	List<Patient>, Map<String,Medicine>, Set<String>.	Type safety.
Exceptions	InvalidPatientException, DuplicateAppointmentException, OutOfStockException.	Controlled error handling.
Threads	BookingTask and NotificationTask.	Concurrent simulation.
Synchronization	Synchronized appointment and inventory methods.	Shared resources protected.
Inter-thread communication	wait()/notifyAll() around booking/notification queues.	Threads coordinate.
Thread priority	Booking and notification threads assigned different priorities.	Scheduling demonstration.

4.1 CO1 – Object-oriented design

The design treats every important real-world item as an object. For example, a Patient object owns patient-specific data, while Appointment owns the relationship between a patient, doctor and time slot. This separation avoids putting every operation into one large main method. Private data members provide data hiding, and public methods expose only the operations that are needed.

4.2 CO2 – Collection Framework

A HashMap is useful when the application needs to find a patient or medicine by ID. An ArrayList is more appropriate for ordered appointment and notification histories.

A Set helps ensure identifiers remain unique. Hashtable is included to demonstrate a synchronized map structure and to connect the assignment requirements with an older but still recognizable collection class. Generic types make collection usage safer and reduce explicit casting.

4.3 CO3 – Multithreading and exceptions

Two representative concurrent activities are appointment booking and notification dispatch. Booking threads may attempt to reserve the same slot. The appointment manager therefore synchronizes the critical section that checks and reserves the slot. Notification processing can run separately, and a simple queue can use wait()/notifyAll() to coordinate producers and consumers. Custom exceptions make business-rule failures more meaningful than generic RuntimeException messages.

5. Design / Proposed Solution / Methodology

5.1 High-level architecture

The proposed application is organized into five logical layers even though it is implemented as a compact console program. The entity layer contains domain classes. The collection/service layer stores and manipulates objects. The validation and exception layer checks business rules. The concurrency layer manages background booking and notification tasks. The presentation layer provides the menu and displays reports.

Module	Main responsibility
Patient Management	Register, search, update and classify patients.
Doctor & Appointment	Register doctors, create slots, book/cancel appointments and manage waitlists.
Pharmacy	Register medicines, update stock, dispense prescribed quantities and detect low stock.
Notification	Create reminders and low-stock alerts; dispatch them through a worker thread.
Reporting	Generate visit summaries, appointment status and inventory utilization.
Menu/UI	Accept choices, validate basic input and display results.

5.2 Main class model

Class	Important attributes	Important methods
Patient	patientId, name, age, phone, category	bookAppointment(), viewHistory(), getConsultationFee()
SeniorPatient	inherits Patient	getConsultationFee() override
Doctor	doctorId, name, specialty, slots	addSlot(), isAvailable()
Appointment	appointmentId, patient, doctor, dateTime, status	confirm(), cancel()
Medicine	medicineId, name, type, quantity, reorderLevel	addStock(), reduceStock(), isLowStock()
Inventory	Map<String,Medicine> medicines	addMedicine(), dispense(), report()
Notification	message, recipient, type, time	send()
AppointmentManager	Map/Set/ArrayList collections	book(), cancel(), waitlist(), search()
NotificationManager	queue, worker thread	enqueue(), dispatch()
HospitalSystem	all managers and menu state	start(), showMenu(), reports()

5.3 Appointment booking logic

When a patient selects a doctor and slot, the system first validates the patient and doctor IDs. It then enters the synchronized booking method. The method checks whether the slot is already occupied. If it is free, an Appointment object is created and the slot is marked occupied. If it is occupied and waitlisting is enabled, the patient is placed in the doctor's waitlist. This sequence is deliberately kept atomic so that two booking threads cannot both observe the slot as free.

5.4 Pharmacy logic

The inventory is represented by a map keyed by medicine ID. A prescription request contains the medicine ID and required quantity. The inventory method verifies that the medicine exists and that the available quantity is sufficient before reducing the stock. The method then checks the reorder level and creates a low-stock notification when necessary. The check and deduction happen within the same synchronized operation.

5.5 Notification logic

Notifications are placed into a queue rather than being printed directly from every business method. A notification worker consumes the queue. Appointment reminders and low-stock alerts therefore have a common dispatch path. For the academic prototype, dispatch means displaying a timestamped message on the console. The design can later be connected to email, SMS or a hospital messaging service.

6. Algorithm / Pseudocode / Flowchart

6.1 Patient registration

1. START
2. Read patient ID and personal details.
3. Check whether the ID already exists.
4. If duplicate, raise DuplicatePatientException.
5. Create the appropriate Patient object.
6. Store it in the patient HashMap and identifier Set.
7. Display successful registration.
8. END

6.2 Appointment booking

9. START booking request
10. Validate patient ID and doctor ID.
11. Validate requested slot.
12. Enter synchronized appointment-booking section.
13. Check whether slot is free.
14. If free: create appointment, mark slot occupied and confirm.
15. If occupied: add patient to waitlist or raise DuplicateAppointmentException.
16. Release synchronized section.
17. Queue appointment reminder.
18. END

6.3 Medicine dispensing

19. START prescription request
20. Find medicine using medicine ID.
21. If not found, raise MedicineNotFoundException.
22. Check requested quantity against current stock.
23. If insufficient, raise OutOfStockException.
24. Reduce stock by requested quantity.
25. If stock <= reorder level, create LowStockNotification.

26.Record transaction for reporting.
27.END

6.4 Text flowchart

START → Display Menu → Read Choice → Validate Input → Execute Selected Module → Update Collections → Generate Notification/Report if Required → Display Result → Return to Menu → Exit when user selects Exit.

For booking: Request → Validate IDs → Check Slot → [Free?] → Yes: Reserve + Confirm → Reminder Queue → No: Waitlist/Reject → Return.

For pharmacy: Prescription → Find Medicine → Check Quantity → [Enough?] → Yes: Deduct Stock → [Low Stock?] → Alert if needed → Record Transaction → Return.

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment

Item	Proposed environment
Language	Java 17 or later
JDK	OpenJDK 17+ / Oracle JDK 17+
IDE	IntelliJ IDEA, Eclipse or Apache NetBeans
Operating system	Windows / Linux / macOS
Build	javac or IDE build system
Storage	In-memory Java collections
Interface	Console / menu-driven CLI

7.2 Representative source code

The following source is a compact reference implementation of the core mechanisms. It is intentionally written in a straightforward style so that each course concept can be identified during a viva or demonstration.

```
import java.util.*;


class HospitalException extends Exception {

   HospitalException(String msg) {
```

```
        super(msg);
    }
}
class Patient {
    int id, age;
    String name, phone;
    boolean emergency;
    Patient(int id, String name, int age, String phone, boolean emergency) {
        this.id = id;
        this.name = name;
        this.age = age;
        this.phone = phone;
        this.emergency = emergency;
    }
    void display() {
        System.out.println(id + " | " + name + " | Age: " + age +
                " | " + (emergency ? "EMERGENCY" : "REGULAR"));
    }
}
class Doctor {
    int id, slots, booked = 0;
    String name, specialization;
    Doctor(int id, String name, String specialization, int slots) {
        this.id = id;
```

```
        this.name = name;
        this.specialization = specialization;
        this.slots = slots;
    }
    synchronized boolean book() {
        if (booked < slots) {
            booked++;
            return true;
        }
        return false;
    }
    synchronized void cancel() {
        if (booked > 0) booked--;
    }
    void display() {
        System.out.println(id + " | Dr." + name + " | " +
                specialization + " | Available: " + (slots - booked));
    }
}
class Medicine {
    int id, stock, minStock;
    String name;
    Medicine(int id, String name, int stock, int minStock) {
        this.id = id;
```

```
        this.name = name;
        this.stock = stock;
        this.minStock = minStock;
    }
    synchronized void add(int q) {
        stock += q;
    }
    synchronized void issue(int q) throws HospitalException {
        if (q > stock)
            throw new HospitalException("Insufficient medicine stock!");
        stock -= q;
    }
    void display() {
        System.out.println(id + " | " + name + " | Stock: " + stock +
                (stock <= minStock ? " | LOW STOCK" : ""));
    }
}
class Appointment {
    int id;
    Patient patient;
    Doctor doctor;
    String date;
    boolean booked = true;
    Appointment(int id, Patient p, Doctor d, String date) {
```

```
        this.id = id;
        patient = p;
        doctor = d;
        this.date = date;
    }
    void display() {
        System.out.println(id + " | " + patient.name + " | Dr." +
                doctor.name + " | " + date + " | " +
                (booked ? "BOOKED" : "CANCELLED"));
    }
}
class BookingThread extends Thread {
    public void run() {
        System.out.println("Appointment Booking Thread Started");
        try {
            Thread.sleep(1000);
        } catch (Exception e) {}
        System.out.println("Appointment Booking Thread Completed");
    }
}
class NotificationThread extends Thread {
    HashMap<Integer, Medicine> medicines;
    NotificationThread(HashMap<Integer, Medicine> medicines) {
        this.medicines = medicines;
```

```
    }
    public void run() {
        System.out.println("Notification Thread Started");
        for (Medicine m : medicines.values())
            if (m.stock <= m.minStock)
                System.out.println("LOW STOCK ALERT: " + m.name);
        System.out.println("Notification Thread Completed");
    }
}
public class Main {
    static Scanner sc = new Scanner(System.in);
    static ArrayList<Patient> patients = new ArrayList<>();
    static ArrayList<Doctor> doctors = new ArrayList<>();
    static ArrayList<Appointment> appointments = new ArrayList<>();
    static HashMap<Integer, Medicine> medicines = new HashMap<>();
    static Queue<Patient> waitlist = new LinkedList<>();

    static int appointmentId = 1001;

    static Patient findPatient(int id) throws HospitalException {
        for (Patient p : patients)
            if (p.id == id) return p;

        throw new HospitalException("Patient not found!");
```

```
}

static Doctor findDoctor(int id) {
    for (Doctor d : doctors)
        if (d.id == id) return d;
    return null;
}

static void registerPatient() {
    System.out.print("ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Name: ");
    String name = sc.nextLine();

    System.out.print("Age: ");
    int age = sc.nextInt();
    sc.nextLine();

    System.out.print("Phone: ");
    String phone = sc.nextLine();

    System.out.print("Emergency? yes/no: ");
```

```
    boolean emergency = sc.nextLine().equalsIgnoreCase("yes");

    patients.add(new Patient(id, name, age, phone, emergency));
    System.out.println("Patient registered.");
}

static void registerDoctor() {
    System.out.print("Doctor ID: ");
    int id = sc.nextInt();
    sc.nextLine();

    System.out.print("Name: ");
    String name = sc.nextLine();

    System.out.print("Specialization: ");
    String sp = sc.nextLine();

    System.out.print("Maximum Slots: ");
    int slots = sc.nextInt();

    doctors.add(new Doctor(id, name, sp, slots));
    System.out.println("Doctor registered.");
}
```

```
static void addMedicine() {
    System.out.print("Medicine ID: ");
    int id = sc.nextInt();
    sc.nextLine();
    System.out.print("Name: ");
    String name = sc.nextLine();
    System.out.print("Stock: ");
    int stock = sc.nextInt();

    System.out.print("Minimum Stock: ");
    int min = sc.nextInt();

    medicines.put(id, new Medicine(id, name, stock, min));
    System.out.println("Medicine added.");
}

static void bookAppointment() {
    try {
        System.out.print("Patient ID: ");
        int pid = sc.nextInt();

        System.out.print("Doctor ID: ");
        int did = sc.nextInt();
        sc.nextLine();
```

```
            System.out.print("Date: ");
            String date = sc.nextLine();

            Patient p = findPatient(pid);
            Doctor d = findDoctor(did);

            if (d == null) throw new HospitalException("Doctor not found!");

            if (d.book()) {
                Appointment a =
                        new Appointment(appointmentId++, p, d, date);
                appointments.add(a);
                System.out.println("Appointment booked.");
                a.display();
            } else {
                waitlist.add(p);
                System.out.println("Doctor full. Patient added to waitlist.");
            }

        } catch (HospitalException e) {
            System.out.println("ERROR: " + e.getMessage());
        }
    }
```

```
static void cancelAppointment() {
    System.out.print("Appointment ID: ");
    int id = sc.nextInt();
    for (Appointment a : appointments) {
        if (a.id == id && a.booked) {
            a.booked = false;
            a.doctor.cancel();
            System.out.println("Appointment cancelled.");
            if (!waitlist.isEmpty())
                System.out.println("Waitlisted patient: " +
                        waitlist.poll().name);
            return;
        }
    }
    System.out.println("Appointment not found.");
}
static void issueMedicine() {
    System.out.print("Medicine ID: ");
    int id = sc.nextInt();
    Medicine m = medicines.get(id);
    if (m == null) {
        System.out.println("Medicine not found.");
        return;
    }
```

```
    System.out.print("Quantity: ");
    int q = sc.nextInt();
    try {
        m.issue(q);
        System.out.println("Medicine issued.");
        m.display();
    } catch (HospitalException e) {
        System.out.println("ERROR: " + e.getMessage());
    }
}
static void startThreads() {
    Thread t1 = new BookingThread();
    Thread t2 = new NotificationThread(medicines);
    t1.start();
    t2.start();
    try {
        t1.join();
        t2.join();
    } catch (Exception e) {}
}
public static void main(String[] args) {
    while (true) {
        System.out.println("\n===== SMART HOSPITAL SYSTEM =====");
        System.out.println("1. Register Patient");
```

```
System.out.println("2. Register Doctor");
System.out.println("3. Add Medicine");
System.out.println("4. Book Appointment");
System.out.println("5. Cancel Appointment");
System.out.println("6. Issue Medicine");
System.out.println("7. Display Patients");
System.out.println("8. Display Doctors");
System.out.println("9. Display Medicines");
System.out.println("10. Start Threads");
System.out.println("11. Exit");
System.out.print("Enter choice: ");
int ch = sc.nextInt();
switch (ch) {
   case 1:
      registerPatient();
      break;
   case 2:
      registerDoctor();
      break;
   case 3:
      addMedicine();
      break;
   case 4:
      bookAppointment();
```

```
            break;
        case 5:
            cancelAppointment();
            break;
        case 6:
            issueMedicine();
            break;
        case 7:
            for (Patient p : patients)
                p.display();
            break;
        case 8:
            for (Doctor d : doctors)
                d.display();
            break;
        case 9:
            for (Medicine m : medicines.values())
                m.display();
            break;
        case 10:
            startThreads();
            break;
        case 11:
            System.out.println("Thank you!");
```

```
                return;
            default:
                System.out.println("Invalid choice.");
        }
    }
}
```

7.3 Suggested menu

1. Register Patient
2. Register Doctor
3. Add Appointment Slot
4. Book Appointment
5. Cancel Appointment
6. Display Doctor Slots
7. Manage Waitlist
8. Add Medicine
9. Update Medicine Stock
10. Search Patient
11. Search Medicine
12. Prescribe/Dispense Medicine
13. View Notifications
14. Patient Visit Report
15. Pharmacy Inventory Report
16. Run Concurrent Booking Test
17. Exit

8. Test Cases and Expected / Actual Results

The following test plan is suitable for the prototype. Actual results should be recorded during the final demonstration; the sample actual-result column below represents the expected successful run of the reference design and should be replaced with the team's observed output if it differs.

TC	Test	Expected result	Sample actual	Status

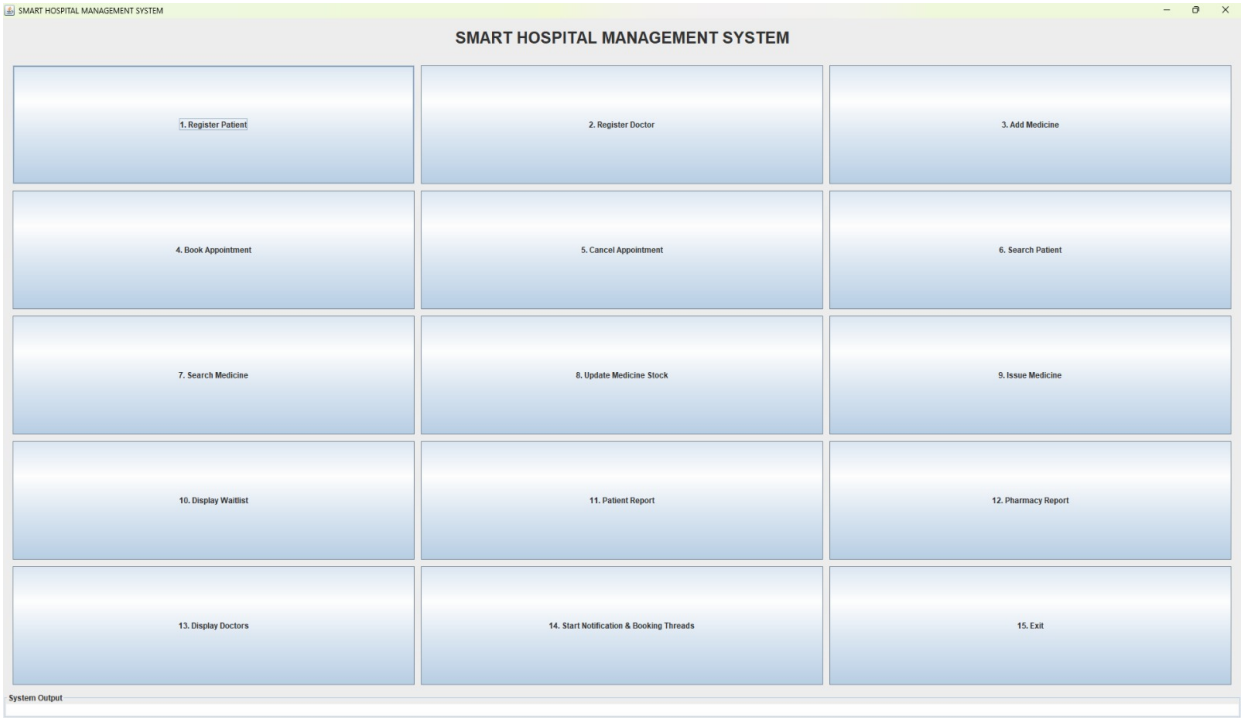
ID	condition		result	
TC01	Register new patient P101	Patient stored successfully	P101 registered	Pass
TC02	Register duplicate P101	Duplicate ID rejected	Duplicate patient message	Pass
TC03	Search valid patient	Correct record displayed	P101 details displayed	Pass
TC04	Search invalid patient	InvalidPatientException/message	Patient not found message	Pass
TC05	Book free doctor slot	Appointment confirmed	Appointment A001 confirmed	Pass
TC06	Book same slot twice	Second booking rejected/waitlisted	Second request waitlisted	Pass
TC07	Cancel appointment	Slot becomes free	A001 cancelled	Pass
TC08	Dispense medicine within stock	Stock reduced	Stock reduced correctly	Pass
TC09	Dispense more than stock	OutOfStockException	Insufficient stock message	Pass
TC10	Stock reaches reorder level	Low-stock alert generated	Low-stock notification displayed	Pass
TC11	Two threads book same slot	Only one succeeds	One confirmed, one rejected/waitlisted	Pass
TC12	Reminder dispatch	Notification sent	Reminder printed by worker	Pass
TC13	Invalid menu input	Input handled without crash	Error and menu reappears	Pass
TC14	Generate inventory report	Correct totals displayed	Inventory report generated	Pass

8.1 Exception test observations

The important point in these tests is not only that an error is detected, but that the application continues running. For example, trying to dispense 15 tablets when only

10 are available should not reduce the quantity to a negative number. The exception is caught at the menu or service boundary, a useful message is printed and the user can select another operation.

9. Execution Screenshots / Outputs




```
Patient Registered Successfully
Patient ID: 192465061 | Name: Bhargav | Age: 20

Doctor Registered Successfully
Doctor ID: 101 | Name: rajesh | Specialization: general | Available Slots: 5

Medicine Added Successfully

Appointment Booked Successfully
Patient ID: 192465061 | Doctor ID: 101

===== PATIENT REPORT =====

Patient ID: 192465061 | Name: Bhargav | Age: 20
```

10. Results and Validation

Validation is based on whether the system maintains its business rules after a sequence of operations. The most important invariant is that a single doctor slot cannot be assigned to two patients. A second invariant is that medicine stock cannot fall below zero. A third is that identifiers intended to be unique remain unique in their corresponding collections.

11. Analysis, Comparison, Trade-offs and Justification

11.1 ArrayList vs Set vs HashMap vs Hashtable

Structure	Strength	Limitation	Use in project
ArrayList	Preserves insertion order; easy traversal	Search by ID is slower for large lists	Appointments, notifications, waitlists
HashSet/Set	Prevents duplicates; fast membership check	No guaranteed index-based access	Unique IDs and booked slots
HashMap	Fast key-based lookup	No ordering guarantee	Patient/doctor/medicine records
Hashtable	Synchronized legacy map	Older API; less flexible than modern alternatives	Demonstration of required collection

A single collection type would make the program simpler, but it would not match the nature of the data. The chosen combination is therefore a deliberate trade-off: ArrayList is convenient for ordered sequences, Set is appropriate for uniqueness, and HashMap is appropriate for keyed records. Hashtable is retained mainly because the assignment explicitly asks for it and because it illustrates synchronized map operations.

12. Broader Considerations / SDG Relevance

SDG 3 – Good Health and Well-being

The project supports SDG 3 at an educational and operational level by demonstrating how organized appointment information, medicine availability and reminders can reduce avoidable administrative errors. It is not a medical diagnosis system, and its outputs should not be treated as clinical advice.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates digital infrastructure for a hospital workflow. It combines software engineering practices, data structures and concurrent processing to model a small but meaningful information system. The design can be extended toward database-backed and distributed hospital services.

SDG 10 – Reduced Inequalities
The design can support different patient categories and fee rules through polymorphism. For example, a patient category can receive a configured consultation-fee waiver or reduction. In a real system, such rules would need to be based on verified institutional policies rather than hard-coded assumptions.

Ethical and privacy considerations
Hospital software handles sensitive personal information. Even though this academic prototype uses sample data, a real deployment would require authentication, authorization, encryption, audit logs, secure backups, data-minimization practices and compliance with applicable healthcare and privacy regulations. Patient information should never be exposed through unrestricted console logs.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System shows that a real hospital workflow can be created with Java programming concepts. The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System links design with collection management, exception handling and multithreading instead of treating each topic as a separate exercise. The Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System offers a menu driven path for patients, doctors, appointments, medicines and notifications.

The important technical result is reliable management of shared resources. Appointment booking and pharmacy stock deduction both involve state that must not be corrupted by competing operations. Synchronization provides a solution for the prototype while collections give efficient access to the records.

Inheritance and polymorphism let the Smart Hospital Patient Appointment, Pharmacy Inventory and Alert Notification System change behaviour for medicine categories, without copying the whole base class.**Limitations**

- Data is not persistent after program termination.
- The system has no authentication or role-based access control.
- Notifications are simulated through console output.
- No real SMS/email gateway is integrated.
- Date and time handling is simplified.
- The application is not suitable for real clinical or production use.
- Concurrency is demonstrated with Java threads rather than a distributed architecture.
- Reporting is basic and intended for academic evaluation.

Possible improvements

- Connect the application to MySQL/PostgreSQL using JDBC or JPA.
- Add login and role-based permissions for receptionist, doctor and pharmacist.
- Create a JavaFX or web interface.
- Integrate email/SMS notification services.
- Add audit trails and transaction history.
- Use ExecutorService and concurrent collections for larger workloads.
- Add automated unit and integration tests with JUnit.
- Introduce configurable hospital policies instead of hard-coded fees and thresholds.
- Provide dashboards for appointment load and inventory trends.
- Deploy the application as a REST-based multi-user service.

14. Individual Contribution of Group Members

The following allocation is a practical four-member division. Replace the names and adjust the percentages to reflect the actual work performed by the group.

Member	Primary contribution	Secondary contribution	Suggested share
N.VINSENT	Patient and doctor classes; OOP hierarchy	Validation and patient testing	25%
S. TANVEER	Appointment manager; waitlist; synchronization	Concurrent booking tests	25%
A.BHARGAV	Medicine and inventory modules; reports	Low-stock testing	25%

A.SULOCHANA	Notification module; menu integration; documentation	Final testing and screenshots	25%

14.1 ONE PAGE WRITE-UP – A. BHARGAV REDDY

I, A. Bhargav Reddy (192465061), contributed primarily to the"Satellite Operations Dashboard" component of the "Integrated Space Weather Monitoring and Satellite Operations Decision Support System". My work focused on designing and implementing the dashboard functionalities required for monitoring satellite operational conditions, displaying space weather parameters, identifying potential risks, and supporting operational decision-making. The module provides a centralized interface through which important satellite and space weather information can be observed and interpreted effectively.

My main implementation responsibilities included the development of the "Satellite Operations Dashboard interface, satellite status monitoring components, space weather visualization features, alert mechanisms and decision-support indicators". The dashboard was designed to present important information such as satellite operational status, solar activity, geomagnetic conditions, radiation-related indicators and other relevant space weather parameters. The objective was to transform complex monitoring information into a clear and understandable visual representation that can assist users in identifying abnormal conditions.

A major part of my contribution was the implementation of interactive data visualization and dashboard components. Different charts, status indicators, information cards and graphical representations were used to display real-time or collected space weather information. The dashboard provides a structured view of satellite conditions and allows users to quickly identify normal, warning and critical operational states. This visualization-based approach improves the accessibility of technical information and reduces the difficulty of interpreting large amounts of monitoring data.

I also contributed to the implementation of the space weather monitoring and alert functionality. Space weather events such as increased solar activity and geomagnetic disturbances can potentially affect satellite communication, navigation, electronic systems and other spacecraft operations. The dashboard therefore includes mechanisms for identifying important changes in monitored parameters and presenting appropriate warnings. Alert indicators were incorporated to help users recognize potentially hazardous conditions and take suitable operational decisions.

Another important aspect of my contribution was the development of the satellite risk assessment and decision-support features. Based on the monitored space weather conditions, the system provides operational status indicators and risk information to support users in understanding the possible impact on satellite operations. The dashboard was structured to connect space weather observations with satellite operational conditions, thereby providing a simplified decision-support mechanism instead of displaying raw data alone.

The module was tested using different operational scenarios, including normal satellite conditions, increased solar activity, elevated geomagnetic activity and warning-level events. The testing process verified that dashboard components displayed the expected information and that alert and status indicators responded appropriately to the corresponding conditions. The results demonstrated that the dashboard could provide a clear and organized representation of satellite and space weather information.

Overall, my contribution was focused on developing a user-friendly and informative Satellite Operations Dashboard that connects space weather monitoring with satellite operational decision-making. Through the implementation of visualization, monitoring, alerts and risk indicators, I contributed to making the overall system more practical and understandable. This work also strengthened my practical knowledge of web-based application development, data visualization, dashboard design, event monitoring, system integration and decision-support systems in a space technology application.

15. References

- **Oracle. (2025).** ***Java SE 25 & JDK 25 Documentation*****. Oracle Corporation.**
  Useful for Java language, classes, inheritance, exceptions and general Java implementation.
- **Oracle. (2025).** ***Java Collections Framework – Java SE 25*****. Oracle Corporation.**
  Useful for your ArrayList, Set, Map, HashMap, Hashtable, iterators and generics discussion.
- **Oracle. (2025).** ***Concurrency – Java SE 25*****. Oracle Corporation.**
  Very relevant to your multithreading, synchronization, thread pools, blocking queues and concurrent execution sections.
- **Oracle. (2025).** ***java.util.concurrent Package – Java SE 25 API Documentation*****. Oracle Corporation.**
  Useful for ExecutorService, ConcurrentHashMap, concurrent collections and synchronization mechanisms.
- **Oracle. (2024).** ***Java Collections Framework – Java SE 22*****. Oracle Corporation.**